Word Roguelike: Code Review and Architecture Plan

Reviewed: 9 GDScript files (about 416 lines) · October 15, 2026

1. The idea behind the game

Your game is Balatro-style score chasing, played by typing. Each in-game day you get a handful of words (currently 5) to type on a physical on-screen keyboard. Every keycap has a point value, a word is worth the sum of its keys, and per-key upgrades bend that total. You must reach the day's required score before you run out of words, or the run ends. The target climbs every day, words can't be reused within a day or a week, and after each day you pick one of three upgrades that attaches to a random plain key.

The design insight that should drive the architecture: **the keyboard is the build.** The keys, their values and their upgrades are the persistent state of a run, the way a deck is in a deckbuilder. Everything else (HUD, letter animations, summary screens) is a *view* of that state. Right now the build lives inside scene nodes, and that is the root of most of the pain below.

2. Summary

Verdict: a solid prototype with good instincts. An `Upgrade` class with an `on_played` hook is exactly the right way to get custom per-upgrade logic (not just flat stats), signals keep typing and scoring loosely coupled, and `WordState` is a clean enum. The project is now at the size where structure, not features, is the bottleneck, as you suspected.

The five findings that matter most:

1. **Run data lives in scene nodes.** The typed buffer is an `Array[Keycap]` of Node instances, and scoring reads `key.value` and `key.upgrade` straight off them. When the keyboard scene is rebuilt, the upgrades and buffer go with it. This is also why `reload_current_scene()` hurts.
2. **`GameState` does four jobs:** it stores run data, scores words, enforces win/lose rules, and advances the calendar.

3. **`WordInput` and `GameState` depend on each other.** `GameState._ready` connects to `WordInput`, and `WordInput.get_word_state` reads `GameState`. It only works because of autoload order.
4. **The view mutates the model.** `WordDisplay` clears `WordInput`'s buffer, and `UpgradeSelectionCard` picks a keycap and applies the upgrade itself.
5. **Game flow is implicit.** Nothing owns the question: what phase are we in? So input is never gated, `paused = false` is duplicated in two screens, and `GameState` emits `day_completed` synchronously inside the submit handler, before any submit animation could finish.

A few things will fail or misbehave today (Section 4). Fix those first; they are small.

Scope note: I did not receive `Upgrade` and its eight subclasses, `Keycap`, `Keyboard`, `ValidWords`, or the main HUD/scene scripts. Findings about those are inferred from how the nine files call them.

3. What's already working

- **Upgrade-as-object.** `key.upgrade.on_played(key, state)` is polymorphism doing its job: each upgrade owns its own logic. Keep this; the plan below only gives it a better context object.
- **Signals for events** (`word_changed`, `word_submitted`, `day_started`, `day_completed`, `day_failed`) instead of direct calls. Most of the plan is extending this habit.
- **Defensive copy.** `buffer.duplicate()` before emitting `word_submitted` prevents listeners from seeing a buffer that changes under them.
- **`WordState` enum** (`VALID`, `USED`, `INVALID`) is a good model of the typing feedback and maps cleanly to colors.
- **`day_completed.gd` feel.** The slam-in tween with random tilt is a nice piece of juice, and `class_name` on `LetterLabel`, `WordDisplay` and `UpgradeSelectionCard` makes the code readable.
- **`game_over.gd`** (5 lines) is the right size for a screen script: show, forward one decision, done.

4. Fix first: things that break or misbehave

#	Where	Problem	Fix
1	`word_display.gd` lines 47 and 88	`animate_submission` is defined twice. GDScript treats this as a compile error.	Delete the first copy and move the whole `NEEDS MIGRATING` block out (Phase 5).
2	`game_state.gd:28`, `word_display.gd:73`	`WordInput.buffer_to_string()` is called but is not defined in the uploaded `word_input.gd` (only `get_word_string()` exists).	If your local copy has it, ignore this. Otherwise add it or call `get_word_string()`.
3	`game_state.gd:80`	`words_used_this_week.assign(words_used_today)` replaces the dictionary's contents in Godot 4.4+, so only the previous day's words stay blocked.	Use `words_used_this_week.merge(words_used_today)`.
4	`game_state.gd:88-91`	The weekly list is only cleared in `reset_game()`, never at the Sunday to Monday rollover. Fixing #3 will expose this: words stay banned forever.	Clear it inside the `current_day == Day.SUNDAY` branch.
5	`upgrade_selection_card.gd:21`	`while keycap.upgrade:` loops forever once every keycap has an upgrade. Each card also rolls its key independently, so two cards can offer the same key.	Filter to keys without upgrades, then pick; handle the empty case.
6	`upgrade_selection_card.gd:27`	Releasing any mouse button (wheel, right-click) picks the upgrade, and nothing stops a second pick.	Check `button_index == MOUSE_BUTTON_LEFT` and disable the card after `picked`.
7	`word_input.gd`	Typing and backspace stay live while a submission animates, and the buffer is cleared afterwards by the view, so keystrokes made in that window are lost.	Add a phase gate (Phase 4).
8	`game_state.gd:20`	`keycap_upgrades` is declared and never written or read.	Becomes `KeyData.upgrade` in Phase 1; delete the dictionary.

5. File-by-file review

`game_state.gd` (autoload, 95 lines)

Does today: stores run data, owns signals, scores words, checks win/lose, advances days and weeks, resets.

- Scoring runs through a stringly-typed dictionary (`state['points']`) that also hands upgrades live references to `words_used_today` and `words_used_this_week`. An upgrade could mutate your history by accident, and a typo in a key name fails silently.

- Points are summed first, then upgrades run in key order. If any upgrade multiplies the running total, the result depends on letter position. Worth checking in your upgrade classes.
- Magic numbers are scattered: `30`, `5` and `1.5` each appear in several functions (`words_remaining = 5` is written three times).
- `required_score * 1.5` compounds with integer truncation: **30, 45, 67, 100, 150, 225, 337** for Monday to Sunday, then **505** and **757** for the next Monday and Tuesday. Unless earning power grows at the same rate, the run hits a wall early in week 2. Make this a tunable curve.
- `day_completed` fires the instant the score is reached, so leftover words are wasted. That is a fine rule, but it rules out a leftover-word bonus later unless you decide on purpose.
- Verdict: **split** into `RunState` (data and rules), `ScoringEngine`, and `GameFlow` (Section 8).

`word_input.gd` (autoload, 57 lines)

Does today: edits the buffer, validates words, gates submission, builds the word string.

- The header says *logical representation, all backend*, but it reads `GameState` and `ValidWords`, and `GameState` reads it back (circular).
- `get_word_state()` rebuilds the string and reruns every check on each keystroke and again on submit. The dictionary lookup runs before the cheap length check (`<= 2`).
- The limits `24` and `2` are hard-coded.
- `type_letter(keycap)` is untyped and the buffer holds Nodes (finding 1).
- Verdict: **split** into `WordBuffer` (text and keys), `WordValidator` (rules, takes history as an argument, which breaks the cycle), and input routing in `GameFlow`.

`keybinds.gd` (autoload, 16 lines)

- Gameplay input (`backspace`, `submit`) and app-level input (quit, fullscreen) share one node. `ui_cancel` quits instantly; you will want Escape to mean Pause soon.
- Nothing checks the game phase, so backspace and submit work on the summary and upgrade screens too (if the node processes while paused).
- Verdict: keep the file small, **split by audience**: `App` (window, quit, fullscreen) and a typing input router that calls `GameFlow`.

`word_display.gd` (98 lines, two scripts in one)

- **Lines 1-42** are a legitimate view: they listen to `word_changed` and show colored letters.
- **Lines 44-98** are HUD orchestration you already marked for migration. They touch `%Score`, `%Bar`, `%SubmittedWords` and `%WordsRemainingLabel`, which belong to the parent HUD. `$WordDisplay` would look for a child with its own name, and `letters[i].keycap` doesn't exist (`LetterLabel` never stores it).
- The view calls `WordInput.clear_buffer()`, so it writes to the model.
- `highlight_keycaps()` awaits each flash in sequence; the comment says it isn't awaited, which is only true at the call site.
- `match state` has no default branch, so `color` is uninitialized if the enum grows. Colors are hard-coded.
- Freed labels stay in `get_children()` until the end of the frame, so a delete and a type in the same frame can miscount.
- **Your label TODO (labels slamming into place):** children of a Container don't own their position; the container does. Tweening `position:y` to `-height` and `0` fights the layout and moves to absolute coordinates instead of an offset. Wrap each letter in a plain `Control` (the container lays out the wrapper) and animate the inner label, or animate `scale` instead.
- Verdict: **split** into `WordDisplay` (letters only), `Hud`, `ScorePanel`, `SubmittedWordsList` and a `SubmissionAnimator`.

`letter_display.gd` (18 lines)

- `set_letter` takes a whole `Keycap` just to read `target_key`. Elsewhere the letter comes from `keycap.get_key()`, so there are two sources of truth for the same character.
- Font size is set in code on every label; put it in a Theme or the label's scene.
- The file is named `letter_display.gd` but defines `LetterLabel`. Match file and class names.

`day_completed.gd` (64 lines)

- Reaches into `GameState` and builds rows with `get_node('Word')` string paths. Give `score_row.tscn` its own script with `set_row(word, points)`.
- `display_completed_day()` is a long async routine with no guard; the Continue button works mid-reveal and the timer chain keeps running.

- `_on_button_pressed` unpauses the tree and calls `GameState.load_next_day()` directly. That is a flow decision living in a view, duplicated in `game_over.gd`.

`game_over.gd` (5 lines)

- Good size. Only issue is the duplicated unpause and direct `reset_game()` call.

`upgrade_selection.gd` (33 lines)

- The upgrade pool is a hard-coded array of classes created with `.new()` on each roll. The same upgrade can appear on all three cards, and there are no weights or rarity.
- `randomize()` on every roll is unnecessary in Godot 4 (the generator is seeded at startup) and makes seeded runs impossible.
- `var card_scene = preload(...)` should be `const`. The `upgrades` list should be shared, not rebuilt per screen instance.
- The screen decides what to offer. That is a model decision living in a view.

`upgrade_selection_card.gd` (30 lines)

- Finds the keyboard through a group and `get_node('Keycaps')`, picks the key, applies the upgrade, then emits `picked`. A card should render an offer and emit `picked(offer)`, nothing else.
- No hover or focus state, so controller and keyboard navigation would need a rewrite. A `Button` base gives this for free.

6. Efficiency notes

None of these will cost you frames at this scale; the real cost is maintenance and correctness. Still worth tidying:

- `WordInput.get_word_state()` recomputes the string and re-queries dictionaries on every call. Cache the text in the buffer and update it on push/pop; run cheap checks (length) before the dictionary lookup.
- `WordDisplay._update_buffer` calls `get_children()` and `%WordDisplay` lookups repeatedly inside loops, and re-colors every letter on every keystroke. Cache the node reference and only recolor when the state changes.
- `highlight_keycaps()` and `animate_submission()` run sequential awaits where one tween with delays would do, and the two stack on top of each other.

- `words_used_today` and `words_used_this_week` are dictionaries used as sets, with the points stored only for one of them. One `WordHistory` object with a set plus an ordered list covers display and lookups.
- `randomize()` in loops and per roll; `preload` assigned to `var`; `upgrades` array rebuilt per instance. Use `const` and one `RandomNumberGenerator`.
- Untyped variables and parameters (`var word =`, `type_letter(keycap)`, `buffer: Array`) hide errors the editor could catch. Turn on the untyped-declaration warning in Project Settings.

7. Why this keeps happening

All nine files share the same few root causes, which is good news: a handful of structural moves fix most of the list.

- **Model in nodes.** Game data (`Keycap`) is a scene object, so scoring depends on the scene tree.
- **God autoload.** `GameState` and `WordInput` accumulate every new rule because they are globally reachable.
- **Circular dependencies.** `WordInput` and `GameState` import each other, which makes both impossible to test alone.
- **Wrong direction of control.** Views change the model (`clear_buffer`, `apply_upgrade`, `load_next_day`) instead of reporting what the player did.
- **Implicit flow.** There is no single place that says what comes after submit, after a day is won, or after an upgrade is picked.
- **Untyped data and magic numbers** instead of small classes and a config resource.

8. Target architecture

Layers and the one rule

```
ui/      ->  flow/  ->  core/
(views)      (orchestration)   (pure game logic)
```

Dependencies point one way. `core/` never touches nodes or the scene tree. `ui/` only reads state and listens to events; player actions go back as signals or one-line calls into `flow/`.

Proposed folders

```
res://
  core/                    pure logic: RefCounted and Resource only
    keys/         key_data.gd, keyboard_model.gd
    typing/       word_buffer.gd, word_validator.gd, word_dictionary.gd
    scoring/      scoring_engine.gd, score_context.gd, score_result.gd,
score_event.gd
    run/          run_state.gd, run_config.gd, word_history.gd
    upgrades/     upgrade.gd, upgrade_registry.gd, upgrade_offer.gd,
                  offer_generator.gd, defs/ (times_two.gd, bookends.gd, ...)
  flow/
    game_flow.gd           autoload: phases and orchestration
    app.gd                 autoload: quit, fullscreen, pause menu later
  ui/
    hud/          hud.gd, word_display.gd, letter_label.gd/.tscn,
                  score_panel.gd, submitted_words_list.gd,
submission_animator.gd
    keyboard/     keyboard_view.gd, keycap_view.gd
    screens/      day_completed/, upgrade_selection/, game_over/
  data/           run_config.tres, word lists
```

Who owns what

Class	Owns	Never does
`KeyData`	letter, base value, equipped upgrade	knows about nodes or animation
`KeyboardModel`	the set of `KeyData`, which keys are free	draw anything
`WordBuffer`	typed keys and cached text, 24-letter cap from config	validate or score
`WordValidator`	rules: dictionary, length, used today or this week	change state
`ScoringEngine`	turns keys + run into a `ScoreResult`	mutate the run or touch UI
`RunState`	day, week, score, words left, history, keyboard; win/lose rules; `apply(result)`	animate or emit UI signals

Class	Owns	Never does
`RunConfig` (Resource)	30, 5, 1.5, 24, 2, week length, difficulty curve	hold run data
`GameFlow`	phase, input gating, sequencing, signals	calculate points
`Upgrade` subclasses	their own effect, through hooks	read the scene tree
Views	show state, play animation, forward clicks	decide anything

Key sketches

Data separated from nodes (fixes the scene-reload problem and replaces `keycap_upgrades`):

```
# core/keys/key_data.gd
class_name KeyData extends RefCounted

var letter: String
var base_value: int = 1
var upgrade: Upgrade = null
```

The `Keycap` node then holds `var data: KeyData` and just renders it. The buffer holds `KeyData`, so a freed node can never break scoring. A new day calls `keyboard_view.refresh()`; nothing reloads.

Upgrade hooks (the custom-logic pattern you already use, with more hook points):

```
# core/upgrades/upgrade.gd
class_name Upgrade extends RefCounted

var id: StringName
var display_name: String
var description: String
var color: String = '#ffffff'

## Called for each key in the word, left to right.
func on_key_scored(_ctx: ScoreContext, _index: int) -> void:
    pass

## Called once after every key has scored (word-wide effects).
func on_word_scored(_ctx: ScoreContext) -> void:
    pass

func on_day_started(_run: RunState) -> void:
    pass
```

Typed scoring context (replaces the `state` dictionary and gives the UI an ordered list of what happened, so it can animate each step):

```
# core/scoring/score_context.gd
class_name ScoreContext extends RefCounted

var word: String
var keys: Array[KeyData]
var run: RunState          # read-only by convention
var chips: int = 0         # additive pool
var mult: float = 1.0      # multiplicative pool
var events: Array[ScoreEvent] = []

func add_chips(amount: int, source: KeyData) -> void:
    chips += amount
    events.append(ScoreEvent.new(source, ScoreEvent.Kind.ADD, amount))

func multiply(factor: float, source: KeyData) -> void:
    mult *= factor
    events.append(ScoreEvent.new(source, ScoreEvent.Kind.MULT, factor))

func total() -> int:
    return roundi(chips * mult)
```

Separating additive and multiplicative pools (the Balatro approach) removes letter-order surprises: a multiplier gives the same result no matter where in the word its key sits.

Flow with phases (fixes input gating, ordering and the duplicated unpause logic):

```
# flow/game_flow.gd (autoload)
extends Node

signal phase_changed(phase: Phase)
signal buffer_changed(text: String, status: WordValidator.Status)
signal word_scored(result: ScoreResult)

enum Phase { TYPING, RESOLVING, DAY_SUMMARY, UPGRADE_PICK, GAME_OVER }

var run: RunState
var buffer := WordBuffer.new()
var phase := Phase.TYPING

func type_key(key: KeyData) -> void:
    if phase != Phase.TYPING:
        return
    buffer.push(key)
    buffer_changed.emit(buffer.text(), WordValidator.check(buffer.text(),
run))

func submit() -> void:
    if phase != Phase.TYPING:
        return
    if WordValidator.check(buffer.text(), run) != WordValidator.Status.VALID:
        return
    _set_phase(Phase.RESOLVING)
    var result := ScoringEngine.score(buffer.keys, run)
    run.apply(result)
    word_scored.emit(result)   # HUD animates, then calls finish_resolving()

func finish_resolving() -> void:
    buffer.clear()
    if run.day_won():
        _set_phase(Phase.DAY_SUMMARY)
    elif run.day_lost():
        _set_phase(Phase.GAME_OVER)
    else:
        _set_phase(Phase.TYPING)
```

One word, end to end

1. A keycap view (click or physical key) calls `GameFlow.type_key(key_data)`.
2. Flow ignores it unless the phase is `TYPING`, pushes to the buffer, emits `buffer_changed`.
3. `WordValidator` returns VALID, USED or INVALID; `WordDisplay` colors the letters.
4. Submit: flow moves to `RESOLVING` and asks `ScoringEngine` for a `ScoreResult` (total plus ordered events).
5. `run.apply(result)` updates score, words left and history.
6. Flow emits `word_scored`; the `SubmissionAnimator` plays the events in order and calls `finish_resolving()`.
7. Flow checks win/lose **after** the animation and moves to `DAY_SUMMARY`, `GAME_OVER` or back to `TYPING`.

Upgrade offers

`OfferGenerator.roll(run, rng, 3)` returns `UpgradeOffer` objects (upgrade plus target `KeyData`), choosing only from keys without an upgrade, without repeating an upgrade or key within one roll, and weighted by the registry. The card receives an offer, shows it, and emits `picked(offer)`. `GameFlow` applies it and advances the phase. The generator uses one `RandomNumberGenerator`, which makes seeded and daily runs possible later.

9. Migration plan

Each phase leaves the game playable, so you can stop after any of them. Size: S is about an hour, M a session or two, L a few sessions.

Phase 0: Stabilize (S)

Apply the eight fixes in Section 4. **Done when** the project compiles, words carry over correctly across days within a week, and the upgrade screen cannot hang.

Phase 1: Separate data from nodes (M)

- Create `KeyData` and `KeyboardModel`; each `Keycap` node references its `KeyData`.
- The buffer, scoring and upgrade application use `KeyData`.

- Delete `keycap_upgrades`; stop depending on `reload_current_scene()` (views refresh from the model instead).

Done when restarting or advancing a day never touches upgrades, and a freed `Keycap` node cannot affect a score.

Phase 2: Scoring engine (M)

- Add `ScoreContext`, `ScoreEvent`, `ScoreResult`, `ScoringEngine`.
- Port the eight upgrades (`TimesTwo`, `BeginsInLetter`, `EndsInLetter`, `ThreeCount`, `DoubleDown`, `SoloStar`, `Bookends`, `FinalStand`) to the hook methods.
- Write a handful of scoring tests (GUT or a plain test script). Upgrades are where bugs will multiply, and `core/` is testable without launching the game.

Done when scoring runs from a plain script with no scene loaded.

Phase 3: Split `GameState` and `WordInput` (M)

- `RunConfig` (Resource) takes the magic numbers; `RunState` holds data and rules (`day_won`, `day_lost`, `apply`, `next_day`, weekly rollover); `WordHistory` replaces the two dictionaries.
- `WordBuffer` (cached text) and `WordValidator` (history passed in, so no autoload cycle).
- Replace the 1.5x compounding with a difficulty curve you can tune per day.

Done when neither `core/` class references an autoload.

Phase 4: `GameFlow` and phases (M)

- Add `GameFlow` with the phase enum and input gating; split `Keybinds` into `App` and an input router.
- Screens call one-liners (`GameFlow.continue_pressed()`, `GameFlow.restart()`); remove the duplicated `get_tree().paused` handling.
- Win/lose is evaluated after the submit animation finishes.

Done when typing during an animation or summary screen does nothing, and no view calls `GameState` directly.

Phase 5: Split the HUD (M)

- `Hud` owns layout and the score/words-left/progress widgets; `WordDisplay` only shows letters; `SubmittedWordsList` gets its own script.
- `SubmissionAnimator` consumes `ScoreEvent`s in order (the Balatro-style tally).
- `LetterLabel` becomes a small scene with a wrapper `Control`, which fixes the slam-into-place TODO.
- Delete the migration block from `word_display.gd`.

Done when `word_display.gd` has no references to score, bar or submitted-words nodes.

Phase 6: Upgrades and screens (M)

- `UpgradeRegistry` (all upgrade types, weights, rarity), `OfferGenerator`, `UpgradeOffer`.
- Dumb `UpgradeSelectionCard`; `ScoreRow` script for the day summary.
- One `RandomNumberGenerator` on the run, with an optional seed.

Done when adding a ninth upgrade means adding one file and one registry line.

Phase 7: Conventions and polish (S)

- File name equals class name in snake_case; one root script per scene; signals named in the past tense.
- Type every variable, parameter and return; enable the untyped-declaration warning.
- No `get_node('A/B')` across scenes; use exported references or signals.
- Colors and fonts in a Theme or `Palette` resource.

10. Room to expand

With this structure these become small additions instead of rewrites:

- **New upgrades:** one file each, registered with a weight. Add hooks as needed (`on_key_typed` for live previews, `on_day_ended`).
- **Day modifiers or boss days:** a `DayModifier` using the same hook pattern as `Upgrade`.
- **Save and load:** `RunState` is plain data, so it can serialize to a dictionary or Resource.
- **Seeded and daily runs:** one RNG on the run.

- **Leftover-word bonus, word-length bonuses, rarity tiers:** scoring and rules live in `core/`, not in UI callbacks.
- **Controller and keyboard navigation:** views are thin and cards can be Buttons.
- **Balance tools:** you can simulate thousands of days from a script, because scoring runs without a scene.

11. Decisions only you can make

1. **Run length:** the `Day` enum is 7 days with a weekly rollover. Is a run a fixed number of days, endless weeks, or does something special happen at the end of a week?
2. **Overkill:** should reaching the target end the day immediately (current behavior), or let the player keep playing leftover words for a bonus?
3. **Upgrade stacking:** can a key hold more than one upgrade, and can an upgrade replace an existing one when all keys are taken?
4. **Failure:** is a failed day always game over, or can a retry or consolation exist?
5. **Scoring semantics:** do multipliers act on the running total, the key's value, or a separate multiplier pool? This decides the shape of `ScoreContext`.

12. Suggested next step

Do Phase 0 now, then Phase 1 before adding any new upgrades. Phase 1 resolves the scene-reload problem and makes everything after it simpler. If you send the `Upgrade` subclasses, `Keycap` and `Keyboard`, I can tailor Phases 1 and 2 to your real code.